

Information Retrieval System — Complete Code Reference

Project: Service-Oriented IR System over the Args.me corpus (Touché 2020 Task 1) **Scope:** Every source file, and every function/class/method within it. **Generated:** 2026-06-25

Part I — System Overview

This project is a complete, modular **Information Retrieval (IR)** system built around a single shared engine library (`ir_core`) that is exposed three ways: a **Streamlit web UI**, a **Service-Oriented Architecture (SOA)** of five FastAPI microservices behind an API gateway, and a set of **command-line scripts** for building indexes and running evaluations.

It retrieves over the **Args.me** debate-argument corpus (387,692 documents, Touché 2020 Task 1), with evaluation queries and relevance judgments (qrels) pulled from the `ir_datasets` library. A tiny 5-document sample corpus ships for smoke tests.

The system implements **six retrieval models** behind one uniform interface:

Model	Representation	Matching
TF-IDF (VSM)	sparse <code>ltc</code> term weights	cosine similarity
BM25	probabilistic Okapi, tunable <code>k1/b</code>	BM25 score
BERT	<code>all-MiniLM-L6-v2</code> sentence embeddings	cosine
Word2Vec	PyTorch skip-gram, mean-pooled word vectors	cosine
Hybrid · Parallel	fuses several models	weighted-sum or RRF
Hybrid · Serial	one model retrieves, another re-ranks	stage-2 score

Part II — Requirements Coverage

The codebase is organized around the project brief's nine graded requirements:

#	Requirement	Where it lives
2	Representations: VSM TF-IDF, BM25, embeddings (BERT and Word2Vec), hybrid (serial and parallel with fusion)	<code>ir_core/representation/</code>

2. The engine lazily loads the dataset **bundle**: the SQLite inverted index, whichever dense vector stores exist, and a query refiner. Bundles are cached so this happens once.
3. If refinement is enabled, the **QueryRefiner** spell-corrects and/or expands the query — keeping only corrections/synonyms that actually occur in the corpus ($df > 0$).
4. **build_context** normalizes the (effective) query with the *same* preprocessing pipeline used to build the index, producing a **QueryContext** of raw text + tokens.
5. The engine dispatches to the requested model: - **TF-IDF / BM25** score by streaming the matching terms' postings from SQLite. - **BERT / Word2Vec** encode the query and run a cosine search over the in-memory (inflated) vector matrix. - **Hybrids** either fuse several models' rankings (parallel) or retrieve-then-rerank (serial).
6. Result **doc_ids** are joined back to their **full original text** (decompressed from the SQLite BLOB) and returned as ranked **SearchResults**.

Part V — Storage & Compression Design

Two storage requirements shape the data layer, and both are satisfied:

- **Original documents live in the database, in full.** The SQLite **docs** table stores each document's complete original text as a **zlib-compressed BLOB** (no truncation). Top-k results are served straight from the DB, and the UI shows the entire document inline.
- **Models and indexes are stored compressed.** Dense vectors are gzip-streamed (***.vectors.f32.gz**), doc-id sidecars are gzipped (***.docids.json.gz**), the Word2Vec matrix is a compressed **.npz**, and its vocab is gzipped JSON. The inverted index's bulky text payload is zlib-compressed inside the DB.
- **Nothing is trained at query time.** TF-IDF and BM25 score from a **pre-built** index (IDF and per-document norms are computed once at build time); the dense vectors and Word2Vec model are built offline and only *loaded* at query time. The first dense query inflates the compressed vector file into memory once, then scoring is fast.

Part VI — Directory Layout

```
ir_core/ # the IR engine (importable library) text/ # preprocessing (documents + queries) data/ #
streaming corpus + qrels/queries index/ # inverted index (SQLite) + dense vector store representation/
# tfidf, bm25, word2vec, embeddings, hybrid query/ # processing + refinement eval/ # metrics + runner
engine.py # orchestrator (Facade) used by services & UI services/ # SOA:
preprocessing/indexing/retrieval/ranking-eval/refinement + gateway ui/app.py # Streamlit UI scripts/ #
build_indexes.py, evaluate.py, run_full_eval.py, smoke_test.py docs/ # ARCHITECTURE.md, REPORT.md,
CODE_REFERENCE.md (root) # preprocessing.py, process_docs.py, check_ready.py, # download_dataset.py,
ir_win_fix.py
```

Part VII — Code Reference: Every File & Function

This part documents every source file and every function, class, method, and HTTP route it contains, grouped by layer.

Section A — Configuration & Shared Types

`ir_core/__init__.py`

Purpose: Package marker for the `ir_core` IR engine library; declares the package docstring and version.

This file contains no functions or classes. It only sets the module docstring (describing the library as a modular IR engine with multiple document representations, disk-backed indexing, query processing/refinement, ranking and evaluation) and defines a single module-level constant `__version__ = "1.0.0"`.

`ir_core/config.py`

Purpose: Central configuration hub: the dataset registry, on-disk index paths, microservice endpoint URLs, and default model/retrieval hyper-parameters, so no path or dataset id is hard-coded elsewhere.

Module-level constants: `PROJECT_ROOT` (the repo root, computed from this file's location), `INDEX_ROOT` (root directory for all persisted indexes, overridable via the `IR_INDEX_ROOT` env var), `DATASETS` (a dict mapping dataset keys to `DatasetSpec` objects, currently just `"argsme"`), `SAMPLE` (a tiny 5-doc `DatasetSpec` for smoke tests), `BERT_MODEL_NAME` (sentence-transformers model name, overridable via `IR_BERT_MODEL`), `DEFAULT_TOP_K = 10`, and `SERVICE_URLS` (a dict of microservice host:port URLs, each overridable via env var).

class DatasetSpec

A frozen dataclass describing one corpus: where its preprocessed docs live and how to fetch its evaluation queries/qrels from `ir_datasets`. Fields: `key: str` (short id used in URLs/CLI/UI), `name: str` (human-friendly name), `processed_json: str` (path to the preprocessed docs JSON array), `ir_datasets_id: str` (id understood by `ir_datasets` for queries + qrels), and `query_text_attrs: tuple` (attribute names to try in order when extracting query text, default `("text", "title", "description", "query")`).

`index_dir(self) -> str`

A read-only property returning this dataset's index directory, computed as `os.path.join(INDEX_ROOT, self.key)`. It does not create the directory; it only composes the path.

class BM25Params

A dataclass holding default BM25 ranking hyper-parameters: `k1: float = 1.5` (term-frequency saturation) and `b: float = 0.75` (length normalization).

class Word2VecParams

A dataclass of default skip-gram Word2Vec hyper-parameters: `dim: int = 100`, `window: int = 5`, `min_count: int = 2`, `negatives: int = 5`, `epochs: int = 3`, `batch_size: int = 1024`, `lr: float = 0.025`, `max_vocab: int = 50000`.

def get_dataset(key: str) -> DatasetSpec

Resolves a dataset key to its `DatasetSpec`. Returns `SAMPLE` if `key` matches the sample key; otherwise looks the key up in `DATASETS`. Raises `KeyError` (listing the known keys) for an unknown key.

def list_datasets() -> list[dict]

Returns a summary dict per registered dataset (the `DATASETS` values plus `SAMPLE`), each with `key`, `name`, and `available` (whether the `processed_json` file exists on disk).

```
def ensure_index_dir(dataset_key: str) -> str
```

Resolves the dataset's index directory, creates it (`os.makedirs(..., exist_ok=True)`), and returns the path. Guarantees the directory exists before artifacts are written.

[ir_core/types.py](#)

Purpose: Defines small shared data types used across the retrieval layer.

class SearchResult

A dataclass representing a single ranked search hit. Fields: `doc_id: str`, `score: float`, `rank: int = 0`, `raw_text: str = ""`, and `components: dict` (default empty, populated by hybrid fusion with per-model component scores for transparency).

```
to_dict(self) -> dict
```

Serializes the result to a JSON-friendly dict. Rounds `score` to 6 decimals, passes through `doc_id/rank/raw_text`, and rounds every `components` value to 6 decimals. Rounding keeps API payloads compact and deterministic.

Section B — Data & Text Layer

[ir_core/data/corpus.py](#)

Purpose: Streaming access to the potentially multi-GB preprocessed corpora and to evaluation queries/qrels from `ir_datasets`; processed JSON arrays are streamed with `ijson` rather than fully loaded.

class Doc

A dataclass representing one preprocessed document. Fields: `doc_id: str`, `tokens: list[str]`, `raw_text: str`.

```
def iter_docs(spec: DatasetSpec, limit: Optional[int] = None) -> Iterator[Doc]
```

Streams documents one at a time from `spec.processed_json` in file order. Opens the file in binary mode and uses `ijson.items(f, "item")` to iterate array records lazily, yielding a `Doc` per record. Fields are read defensively (`rec.get(...)`), `doc_id` falling back to the running index, `tokens/raw_text` to empty. Stops once `n` reaches a truthy `limit` (`None/0` = all docs).

```
def count_docs(spec: DatasetSpec, limit: Optional[int] = None) -> int
```

Counts documents by iterating `iter_docs(...)`, never loading the whole file; `limit` caps the count.

```
def _query_text(query_obj, attrs: tuple) -> str
```

Private helper extracting a usable query string from an `ir_datasets` query object (whose shape varies by corpus). Tries each attribute name in `attrs` in order, returning the first truthy one; falls back to `default_text()` if present; else `""`.

```
def load_queries(spec: DatasetSpec, limit: Optional[int] = None) -> list[tuple[str, str]]
```

Loads evaluation queries as `(query_id, query_text)` tuples. Imports `ir_win_fix` (side effect) then `ir_datasets`, loads the dataset by `spec.ir_datasets_id`, iterates `queries_iter()` extracting text via `_query_text` with the spec's `query_text_attrs`, and stops once `limit` queries are collected.

```
def load_qrels(spec: DatasetSpec) -> dict[str, dict[str, int]]
```

Loads relevance judgments as a nested dict `{query_id: {doc_id: relevance}}`. Imports `ir_win_fix` then `ir_datasets`, iterates `qrels_iter()`, building per-query inner dicts (relevance cast to `int`).

[ir_core/text/preprocessing.py](#)

Purpose: The canonical text-normalization pipeline shared by documents and queries, ensuring query terms and index terms share a term space; reproduces the exact pipeline used to build the corpus JSON.

On import it ensures NLTK resources (`punkt_tab`, `punkt`, `stopwords`, `averaged_perceptron_tagger_eng`, `wordnet_omw-1.4`) are present (downloading missing ones quietly), then builds module singletons: `_STEMMER` (Porter), `_LEMMATIZER` (WordNet), `_STOPWORDS_EN`, `_PUNCT_TABLE`.

```
def _wordnet_pos(treebank_tag: str)
```

Maps a Penn Treebank POS tag to the WordNet POS constant (`J`→adj, `N`→noun, `V`→verb, `R`→adverb; default noun), improving lemmatization.

```
def process(text: str, *, lowercase: bool = True, remove_punctuation: bool = True,
remove_stopwords: bool = True, do_lemmatize: bool = False, do_stemming: bool = True) ->
list[str]
```

The single source of truth for normalizing text into tokens, applied identically to documents and queries. Short-circuits to `[]` for empty input, then optionally lowercases, strips punctuation, word-tokenizes keeping only alphabetic tokens, optionally drops stopwords, optionally lemmatizes using POS tags, and optionally Porter-stems. The defaults (stemming on, lemmatization off) match how the corpus JSON was generated, guaranteeing query/index term-space agreement.

```
def stem(term: str) -> str
```

Stems a single term (lowercase + Porter). Used by query expansion so injected synonyms land in the same stemmed term space as the index.

[ir_core/query/processing.py](#)

Purpose: Implements query processing (requirement #4) by normalizing the raw query with the same pipeline as the documents and packaging it into a `QueryContext`.

```
def build_context(raw_query: str, bm25_k1: Optional[float] = None, bm25_b: Optional[float]
= None) -> QueryContext
```

Builds a `QueryContext`: stores the raw query (coercing falsy to `""`), tokenizes it via `process` with default flags (so query and index terms match), and passes through optional per-query BM25 overrides.

[ir_core/query/refinement.py](#)

Purpose: Implements query refinement (requirement #5) — spell correction, WordNet synonym expansion, query suggestion/auto-completion, and history-based personalization — in a corpus-aware way: corrections and expansion terms are only kept when they occur in the index (`df > 0`).

On import it ensures NLTK `words` and `wordnet` corpora exist, builds `_ENGLISH` (lowercased English word set), and defines `_LETTERS` and `_WORD_RE`.

```
def _edits1(word: str) -> set[str]
```

Generates the full set of strings at edit-distance 1 (Norvig): deletions, transpositions, replacements, and insertions, returned as a set.

class RefinementResult

A dataclass bundling a refinement pass's output. Fields: `original`, `corrected` (after spell-correction), `refined_raw` (corrected + synonym words, for BERT), `refined_tokens` (corrected + expansion tokens, stemmed, for lexical retrieval), `corrections` (`{wrong: right}`), `expansions` (synonym words added), `suggestions` (suggested queries), `applied` (which features ran and their outputs).

```
to_dict(self) -> dict
```

Serializes all eight fields into a plain dict for JSON responses.

class QueryRefiner

Corpus-aware refiner using an `InvertedIndex` to validate/rank candidate corrections, expansions, and suggestions against the actual corpus vocabulary.

```
__init__(self, index: InvertedIndex)
```

Stores the `InvertedIndex`; all methods consult it for document frequencies and vocabulary samples.

```
_best_correction(self, word: str) -> Optional[str]
```

Picks the best spell-correction for one word, or `None`. Declines when the word is already a known English word, ≤ 3 chars, when `_ENGLISH` is empty, or when its stem already exists in the index. Otherwise gathers edit-distance-1 real-word candidates (falling back to distance-2) and chooses the one with the highest corpus `df`, returning it only if that `df > 0` — so corrections are corpus-attested.

```
correct(self, query: str) -> tuple[str, dict]
```

Spell-corrects an entire query via `_WORD_RE.sub` with a per-word callback that substitutes `_best_correction` when a different valid fix is found, recording each change. Returns the corrected query and the corrections dict.

```
expand(self, query: str, max_per_word: int = 2) -> list[str]
```

Adds up to `max_per_word` WordNet synonyms per word. Seeds a `seen` set with original token stems, iterates synsets/lemmas, skips multi-word lemmas and the word itself, stems each candidate, and keeps only candidates whose stem is unseen and whose index `df > 0`. Returns the added synonyms.

```
suggest(self, query: str, history: Optional[list[str]] = None, limit: int = 5) -> list[str]
```

Produces up to `limit` suggestions: first past queries from `history` sharing a processed token with the current query (excluding an exact repeat), then auto-completions of the last word via `index.vocab_sample(...)`. De-duplicates preserving order and truncates to `limit`.

```
history_terms(self, history: Optional[list[str]], query: str, max_terms: int = 3) -> list[str]
```

Extracts a few salient terms from related past queries to bias retrieval. Returns `[]` with no history; otherwise walks history most-recent-first and collects tokens from queries sharing a token with the current query that are not in the current query, not already collected, and present in the index, stopping at `max_terms`.

```
refine(self, query: str, *, do_spell: bool = True, do_expand: bool = True, use_history: bool = True, history: Optional[list[str]] = None, max_per_word: int = 2) -> RefinementResult
```

The one-shot orchestrator. Optionally spell-corrects, then optionally expands synonyms on the corrected text, then optionally derives history terms. Builds `refined_raw` (corrected text + synonym words, for BERT) and `refined_tokens` (processed corrected tokens + stemmed expansions + history terms, for lexical retrieval), always computes `suggestions`, and records what was applied.

Section C — Indexing Layer

`ir_core/index/inverted_index.py`

Purpose: Disk-backed inverted index built on SQLite that streams the preprocessed corpus to disk and serves as the shared substrate (postings, document frequencies, IDF, per-document TF-IDF norms) for both the TF-IDF/VSM and BM25 scorers.

```
def _db_path(spec: DatasetSpec) -> str
```

Returns the absolute path to the SQLite database file (`inverted_index.sqlite`) inside the dataset's index directory.

```
def _compress_text(text: str) -> Optional[bytes]
```

zlib-compresses the full document text (UTF-8, level 6) so the complete, untruncated text is stored compactly as a BLOB. Returns `None` for empty input so no blob is stored.

```
def _decompress_text(value) -> str
```

Inverse of `_compress_text`: zlib-decompresses a BLOB to a string. Tolerates legacy plain-`str` rows (returned as-is) and `None` (returns `""`), and on `zlib.error/OSError` returns `""` so corrupt/old indexes still read.

```
def _connect(path: str) -> sqlite3.Connection
```

Opens a SQLite connection tuned for bulk build and read throughput. Sets `check_same_thread=False` (so the engine can reuse one connection across threads such as Streamlit reruns), WAL journaling, `synchronous=NORMAL`, in-memory temp store, ~200 MB page cache, and registers Python `LOG/SQRT` user functions so SQL-side weighting uses the same base-e log semantics as the Python scorer.

class Posting

A dataclass for one decoded posting joined with doc metadata: `doc_rowid: int`, `tf: int`, `length: int`, `tfidf_norm: float`. Carrying the norm on the posting avoids a second lookup during scoring.

class InvertedIndex

Read/build interface over the SQLite inverted index for one dataset.

```
__init__(self, spec: DatasetSpec)
```

Stores the spec, computes the DB path, and initializes lazy caches (`_conn`, `_num_docs`, `_avgdl`) to `None`. No I/O here.

```
conn(self) -> sqlite3.Connection (property)
```

Lazily opens and caches the read connection (raising `FileNotFoundError` with a build hint if the file is missing).

exists(self) -> bool

Whether the SQLite file is present, without opening a connection. Used by the engine to decide if a dataset is queryable.

close(self)

Closes the cached connection and resets `_conn`, allowing a later reopen. Called before a rebuild deletes the file.

build(self, limit: Optional[int] = None, keep_raw: bool = True, log_every: int = 5000) -> dict

Builds the entire index from scratch by streaming the corpus (idempotent — deletes any existing `.sqlite/-wal/-shm` first). Steps: (1) create `meta`, `docs`, `terms`, `postings` tables; (2) iterate `iter_docs`, assigning each doc a 1-based `rowid`, computing a token `Counter` and length, optionally storing the zlib-compressed raw text, batching `docs/postings` inserts (flushing every 200k postings); (3) compute per-term `df` via one `GROUP BY` over postings; (4) set smoothed `idf = ln(N/df)` in SQL; (5) create B-tree indexes on `postings(term)`, `postings(doc_rowid)`, and a unique `docs(doc_id)`; (6) compute each document's TF-IDF L2 norm in a single linear Python pass over the postings■terms join; (7) write `meta` (`num_docs`, `avgdl`, `total_terms`) and run `ANALYZE`. Returns the `meta` dict.

_meta(self, key: str, cast)

Reads a single `meta` value by key and applies `cast` (e.g. `int/float`), returning `None` if absent.

num_docs(self) -> int (property)

Corpus document count `N`, cached after first read from `meta`. Needed by BM25 IDF and used to compute stored TF-IDF IDF.

avgdl(self) -> float (property)

Average document length in tokens, cached after first read. Used by BM25's length-normalization term `b`.

df(self, term: str) -> int

Document frequency of a term from `terms` (0 if unknown). Used by BM25's IDF.

idf(self, term: str) -> float

Precomputed smoothed IDF (`ln(N/df)`) of a term (0.0 if absent). Used by the TF-IDF/VSM scorer.

postings(self, term: str) -> list[Posting]

All postings for a term as `Posting` objects, joining `postings` with `docs` so each carries `tf`, `length`, and `tfidf_norm`. The inner-loop input for both lexical scorers.

doc_meta(self, term_count: bool = False)

Convenience accessor returning (`num_docs`, `avgdl`). `term_count` is currently unused.

get_docs(self, rowids: Iterable[int]) -> dict[int, dict]

Fetches display metadata for internal `rowids`, returning `{rowid: {"doc_id", "raw_text", "length"}}` with `raw_text` decompressed. Returns `{}` for empty input. Attaches readable text to lexical hits.

get_docs_by_id(self, doc_ids: Iterable[str]) -> dict[str, str]

Maps external `doc_id -> raw_text` (decompressed), chunking the IN-clause in batches of 500. Used by embedding retrievers, which key on `doc_id`.

```
rowids_for(self, doc_ids: Iterable[str]) -> dict[str, int]
```

Maps external `doc_id` -> `internal rowid` (chunked by 500). Converts an embedding candidate set into a rowid filter so a lexical re-scorer can restrict scoring.

```
vocab_sample(self, prefix: str, limit: int = 10) -> list[str]
```

Up to `limit` vocabulary terms starting with `prefix`, ordered by descending df. Supports prefix/autocomplete suggestion.

```
all_terms_with_df(self, min_df: int = 1, limit: Optional[int] = None)
```

(`term`, `df`) rows for all terms with `df >= min_df`, descending by df, optionally capped. Used for corpus stats and refinement vocabularies.

[ir_core/index/vector_store.py](#)

Purpose: Disk-backed dense-vector store for the embedding representations; vectors are streamed to disk as gzip-compressed L2-normalized float32 and, at search time, inflated once into memory and scored by batched matrix-vector products (dot product = cosine).

```
def _paths(spec: DatasetSpec, model: str)
```

Returns the triple of paths for a model: the gzip'd float32 vectors (`emb_<model>.vectors.f32.gz`), the plain JSON meta (`.meta.json`), and the gzip'd doc-id sidecar (`.docids.json.gz`). Meta stays uncompressed so `N/dim` can be read before inflating the vectors.

class VectorStoreWriter

Streams float32 rows to disk during a build; the caller invokes `add` per batch then `close`.

```
__init__(self, spec: DatasetSpec, model: str, dim: int)
```

Ensures the index dir exists, resolves the three paths, records `model/dim`, initializes `n` and the `doc_ids` accumulator, and opens the gzip vector file for binary writing (level 6), holding the handle open across `add` calls.

```
add(self, doc_ids: list[str], vectors: np.ndarray)
```

Appends a batch: coerces to float32, promotes a 1-D vector to a row, L2-normalizes each row (guarding zero-norm) so a later dot product equals cosine, writes raw bytes to the gzip stream, and extends the doc-id list and `n`.

```
close(self)
```

Closes the gzip vector stream, writes the gzip'd JSON doc-id sidecar, and writes the plain JSON meta (`model`, `dim`, `n`).

class VectorStore

Read-only view over a built vector store used for cosine search.

```
__init__(self, spec: DatasetSpec, model: str)
```

Resolves the three paths and initializes lazy caches (`_vectors`, `_doc_ids`, `_meta`) to `None`.

```
exists(self) -> bool
```

True only if all three files (vectors, meta, doc-ids) are present. The engine uses this to decide whether to wire up the dense retriever.

```
meta(self) -> dict (property)
```

Lazily loads/caches the plain JSON meta dict. Read before inflating vectors so the shape is known.

`doc_ids(self) -> list[str] (property)`

Lazily loads/caches the gzip'd JSON list of doc-ids, ordered row-for-row with the vector matrix.

`_id_to_row(self) -> dict[str, int] (property)`

Lazily builds/caches a `doc_id -> row-index` map for subset scoring.

`score_subset(self, query_vec: np.ndarray, doc_ids: list[str]) -> dict[str, float]`

Cosine of the query against a specific subset (for serial-hybrid re-ranking). L2-normalizes the query (returns `{}` on a zero query), resolves requested doc-ids to rows, dot-products those rows, returns `{doc_id: cosine}`. Skips unknown doc-ids.

`vectors(self) -> np.ndarray (property)`

Inflates the gzip'd float32 blob once into a resident `(n, dim)` array via `np.frombuffer(...).reshape(...)`, caching it. Decompression is a one-time first-access cost.

`search(self, query_vec: np.ndarray, top_k: int = 10, batch: int = 100_000) -> list[tuple[str, float]]`

Brute-force cosine top-k. Normalizes the query (returns `[]` if zero), processes the matrix in row batches, reduces each batch's dot products to a candidate top-k via `argpartition`, accumulates, and prunes back to the global top-k after each batch (memory-bounded running top-k). Sorts survivors descending and returns `[(doc_id, score)]`.

Section D — Representation Layer

`ir_core/representation/__init__.py`

Purpose: Package marker documenting that every representation/scoring model exposes a uniform `search(query_tokens, top_k, ...) -> list[SearchResult]` interface so the retrieval layer and hybrid combiner treat them uniformly. Contains only the module `docstring`.

`ir_core/representation/base.py`

Purpose: Defines the uniform retriever interface — a `QueryContext` carrying both the raw query and its tokens, plus `LexicalRetriever/DenseRetriever` wrappers — so lexical and dense models can be driven identically.

class QueryContext

A dataclass carrying everything any retriever needs: `raw: str` (for BERT), `tokens: list[str]` (for TF-IDF/BM25/Word2Vec), and optional per-query BM25 overrides `bm25_k1/bm25_b`.

class LexicalRetriever

Uniform wrapper around the two lexical models (TF-IDF or BM25), both consuming tokens + the shared inverted index.

`__init__(self, name: str, index: InvertedIndex)`

Stores the model `name` and index, and eagerly constructs both a `TfidfVSM` and a `BM25`; `search/rescore` dispatch to the one named by `name`.

```
search(self, ctx: QueryContext, top_k: int = 10) -> list[SearchResult]
```

Dispatches to TF-IDF when `name == "tfidf"`, else BM25 (passing the per-query `bm25_k1/bm25_b`). Returns the ranked list.

```
rescore(self, ctx: QueryContext, doc_ids: list[str]) -> dict[str, float]
```

Re-scores a candidate set (serial hybrid). Resolves doc-ids to rowids via `index.rowids_for`, runs the configured scorer restricted to those rowids, and returns `{doc_id: score}` (or `{}` if none resolve).

class DenseRetriever

Uniform wrapper around an embedding model presenting the same surface as `LexicalRetriever`.

```
__init__(self, name: str, retriever: EmbeddingRetriever)
```

Stores the `name` and the underlying `EmbeddingRetriever` to delegate to.

```
search(self, ctx: QueryContext, top_k: int = 10) -> list[SearchResult]
```

Delegates to the embedding retriever's `search`, passing the `raw` query (encoders work on text, not tokens).

```
rescore(self, ctx: QueryContext, doc_ids: list[str]) -> dict[str, float]
```

Delegates to the embedding retriever's `rescore` with the raw query and candidate doc-ids. Used as the stage-2 re-ranker in serial hybrid.

[ir_core/representation/tfidf.py](#)

Purpose: Vector Space Model with TF-IDF (`ltc`) weighting and cosine similarity (requirement #2, model 1); document vectors are never fully materialized — the dot product is accumulated term-by-term over the postings and divided by the precomputed per-document and query L2 norms.

class TfidfVSM

The TF-IDF/cosine scorer (`name = "tfidf"`).

```
__init__(self, index: InvertedIndex)
```

Stores the shared inverted index (source of IDF, postings, norms).

```
_query_weights(self, query_tokens: list[str]) -> dict[str, float]
```

Computes the query-side `ltc` weights: counts query term frequencies and assigns `weight = (1 + ln tf) * idf(t)` for terms with positive IDF (others dropped).

```
search(self, query_tokens: list[str], top_k: int = 10, candidate_filter: Optional[set[int]] = None) -> list[SearchResult]
```

Computes cosine similarity between the query and every document containing a query term. Builds query weights and norm, then for each query term iterates its postings accumulating `wq * wd` where `wd = (1 + ln tf) * idf`, recording each doc's precomputed `tfidf_norm`. An optional `candidate_filter` restricts scoring to given rowids. Normalizes each dot product to a cosine by `doc_norm * query_norm`, sorts descending, truncates to `top_k`, and attaches display metadata.

[ir_core/representation/bm25.py](#)

Purpose: BM25/Okapi probabilistic scorer (requirement #2, model 3) over the same inverted index as TF-IDF, exposing `k1` (term-frequency saturation) and `b` (length normalization) per query.

class BM25

The BM25/Okapi scorer (`name = "bm25"`).

`__init__(self, index: InvertedIndex, params: Optional[BM25Params] = None)`

Stores the index and default `BM25Params`, which supply `k1/b` when a query doesn't override them.

`_bm25_idf(N: int, df: int) -> float (staticmethod)`

Robertson/Spärck-Jones IDF $\ln(1 + (N - df + 0.5)/(df + 0.5))$. The leading `1 +` keeps IDF non-negative even for very frequent terms.

`search(self, query_tokens: list[str], top_k: int = 10, k1: Optional[float] = None, b: Optional[float] = None, candidate_filter: Optional[set[int]] = None) -> list[SearchResult]`

Scores with the BM25 Okapi formula. Resolves effective `k1/b` (per-query overrides else defaults), reads `N/avgdl`, then for each unique query term with `df > 0` computes its IDF and iterates postings, adding `idf * (tf*(k1+1)) / (tf + k1*(1 - b + b*(length/avgdl)))`. An optional `candidate_filter` restricts to given rowids. Sorts descending, truncates to `top_k`, decorates with display metadata.

[ir_core/representation/embeddings.py](#)

Purpose: The two embedding representations (requirement #2, model 2): a BERT sentence-transformer encoder and the PyTorch Word2Vec encoder, plus build functions that stream the corpus into a `VectorStore`, and the shared `EmbeddingRetriever` that encodes a query and runs cosine search.

class BertEncoder

Lazy wrapper around a sentence-transformers model (`name = "bert"`).

`__init__(self, model_name: str = BERT_MODEL_NAME)`

Stores the model name and defers loading (`_model = None`).

`model(self) (property)`

Lazily imports `sentence_transformers` and instantiates the model on first use, caching it.

`dim(self) -> int (property)`

Returns embedding dimensionality, using `get_embedding_dimension` if present else `get_sentence_embedding_dimension` (handling library version differences).

`encode_texts(self, texts: list[str], batch_size: int = 64) -> np.ndarray`

Encodes a list of texts into a float32 array (progress bar off, numpy out). Used at build time and for queries.

`encode_query(self, text: str) -> np.ndarray`

Encodes a single query via `encode_texts([text])[0]`.

class Word2VecEncoder

Adapts a trained `Word2VecModel` to the encoder interface (`name = "word2vec"`).

`__init__(self, model: Word2VecModel)`

Stores the trained Word2Vec model.

```
dim(self) -> int (property)
```

Returns the model's vector dimensionality.

```
encode_query(self, text: str) -> np.ndarray
```

Preprocesses the raw query with the shared `process` tokenizer and returns the mean of its in-vocabulary word vectors, matching how documents were embedded.

```
def build_bert_index(spec: DatasetSpec, limit: Optional[int] = None, batch_size: int = 256, log_every: int = 2560) -> dict
```

Builds the BERT vector store by streaming the corpus through a `VectorStoreWriter`: batches doc-ids and text (falling back to joined tokens when raw text is missing), encodes each full batch, writes the (L2-normalized at write time) vectors, flushes a trailing partial batch. Returns a summary dict.

```
def build_word2vec_index(spec: DatasetSpec, model: Word2VecModel, limit: Optional[int] = None, batch_size: int = 4096, log_every: int = 20000) -> dict
```

Builds the Word2Vec document vector store: computes each doc's mean-of-token-vectors embedding, `np.vstacks` each batch, streams through a `VectorStoreWriter`, flushes the trailing batch. Returns a summary dict.

```
class EmbeddingRetriever
```

Cosine search over a stored embedding model; the optional inverted index attaches display text to hits.

```
__init__(self, spec: DatasetSpec, model_name: str, encoder, index: Optional[InvertedIndex] = None)
```

Stores the model name, opens a read-only `VectorStore`, and keeps the query encoder and optional inverted index.

```
search(self, query_text: str, top_k: int = 10) -> list[SearchResult]
```

Encodes the query; returns `[]` on an all-zero vector. Otherwise runs `VectorStore.search` for top-k (`doc_id`, `cosine`), fetches raw text for those doc-ids when available, and returns ranked `SearchResults`.

```
rescore(self, query_text: str, doc_ids: list[str]) -> dict[str, float]
```

Encodes the query (returns `{}` on a zero vector) and computes cosine against a candidate subset via `VectorStore.score_subset`. Used as a stage-2 serial-hybrid re-ranker.

[ir_core/representation/word2vec.py](#)

Purpose: A genuine skip-gram Word2Vec model with negative sampling implemented directly in PyTorch (gensim has no Python 3.14 wheel); documents/queries are embedded as the mean of their in-vocabulary word vectors. This is the second embedding model alongside BERT.

```
def _paths(spec: DatasetSpec)
```

Returns the two artifact paths: the compressed `.npz` word matrix (`word2vec.vectors.npz`) and the gzip'd JSON vocab (`word2vec.vocab.json.gz`).

```
class Word2VecModel
```

Holds the trained word matrix plus vocabulary and turns token lists into vectors.

```
__init__(self, vectors: np.ndarray, vocab: dict[str, int], params: Optional[Word2VecParams] = None)
```

Stores the (**V**, **dim**) float32 word matrix (already L2-normalized), the **term** -> **row** vocab, derives **dim**, and keeps/creates **Word2VecParams**.

```
embed_tokens(self, tokens: Iterable[str]) -> np.ndarray
```

Embeds a token list as the mean of its in-vocabulary word vectors. OOV tokens are dropped; if none remain it returns a zero vector. The standard mean-pooling representation for Word2Vec retrieval.

```
most_similar(self, term: str, topn: int = 10) -> list[tuple[str, float]]
```

Returns the **topn** most cosine-similar vocabulary terms to a term (cosine = dot since L2-normalized), excluding the term itself; **[]** for an unknown term.

```
save(self, spec: DatasetSpec)
```

Persists the model: the word matrix as compressed **.npz** and the vocab+**dim** as gzip'd JSON, ensuring the index dir exists first.

```
load(cls, spec: DatasetSpec) -> "Word2VecModel" (classmethod)
```

Loads a saved model: reads **vectors** from the **.npz** and **vocab** from the gzip'd JSON, reconstructing a **Word2VecModel**.

```
exists(cls, spec: DatasetSpec) -> bool (classmethod)
```

Whether both artifact files (**.npz** matrix and gzip'd vocab) are present, so the engine can decide whether to wire up the Word2Vec retriever.

class _SkipGram

A **torch.nn.Module** implementing skip-gram-with-negative-sampling with separate center and context embedding tables.

```
__init__(self, vocab_size: int, dim: int)
```

Creates **center** and **context nn.Embedding** tables (**vocab_size**, **dim**), initializing center weights uniformly in **[-0.5/dim, 0.5/dim]** and context weights to zero.

```
forward(self, centers, contexts, negatives)
```

Computes the negative-sampling loss for a batch: looks up center **c**, positive **pos**, and **k** negatives **neg**; computes **c·pos** and batched **neg·c**; applies **logsigmoid(pos)** and **logsigmoid(-neg).sum**; returns mean NLL **-(pos_loss + neg_loss).mean()**.

```
def train_word2vec(token_stream_factory, params: Optional[Word2VecParams] = None, seed: int = 13, log_every: int = 200) -> Word2VecModel
```

Trains skip-gram over a corpus. **token_stream_factory** is a callable returning a fresh document-token iterator each call, enabling multiple passes. Pass 1 builds the vocabulary (drop below **min_count**, sort by frequency, cap at **max_vocab**; raises **ValueError** if empty). Builds the negative-sampling distribution **freq^{0.75}**, picks CUDA if available, and trains **params.epochs** epochs: a **gen_pairs** generator yields (**center**, **context**) pairs using a randomly shrunk window, buffered to **batch_size** and pushed through **_step**. Finally L2-normalizes the center matrix per row and wraps it in a **Word2VecModel**.

```
def _step(model, opt, centers, contexts, neg_table, k, V, device) -> float
```

One optimization step: moves indices to tensors, draws `len(centers)*k` negatives via `torch.multinomial` reshaped to `(B, k)`, zeroes grads, runs the forward loss, backprops, steps Adam, returns the scalar loss.

[ir_core/representation/hybrid.py](#)

Purpose: The hybrid representation (requirement #2, model 4) implemented both ways — parallel fusion (weighted-sum or Reciprocal Rank Fusion) and serial retrieve-then-rerank — keeping per-model component scores on each result for transparency.

```
def _min_max(scores: dict[str, float]) -> dict[str, float]
```

Min-max normalizes a score dict to `[0, 1]`. Returns `{}` for empty input, maps everything to `1.0` when all scores are equal, else `(v - lo)/(hi - lo)`. Makes heterogeneous model scores comparable before fusion.

```
def parallel_hybrid(retrievers: dict, ctx: QueryContext, top_k: int = 10, fusion: str = "weighted", weights: Optional[dict] = None, pool: int = 100) -> list[SearchResult]
```

Runs every retriever independently (each retrieving `pool` candidates) and fuses the ranked lists. `weights` default to equal. With `fusion == "rrf"` it uses Reciprocal Rank Fusion ($w * 1/(60 + \text{rank})$) per doc, recording each model's contributing rank); otherwise (`weighted`) it min-max normalizes each model's scores and adds the weighted normalized score (recording each model's raw score). Ranks by fused score, truncates to `top_k`, returns `SearchResults` carrying a `components` dict.

```
def serial_hybrid(stage1, stage2, ctx: QueryContext, top_k: int = 10, candidate_k: int = 100) -> list[SearchResult]
```

The retrieve-then-rerank cascade. Stage 1 (cheap, high-recall) retrieves `candidate_k` candidates (returns `[]` if none); stage 2 (expensive, semantic) re-scores only those doc-ids via `rescore`. Both score sets are min-max normalized, and each doc's final score is the stage-2 normalized score plus a tiny `1e-3 * stage-1 tiebreak` (so docs stage 2 cannot score keep stage-1 order). Ranks by stage-2 score, truncates to `top_k`, records both stages in `components`.

[ir_core/engine.py](#)

Purpose: The single retrieval entry point used by the microservices and the Streamlit UI; it lazily loads whichever indexes a dataset has, applies optional query refinement, and dispatches to the requested representation including the two hybrids.

Module constants: `SINGLE_MODELS = ["tfidf", "bm25", "bert", "word2vec"]`, `HYBRID_MODELS = ["hybrid_parallel", "hybrid_serial"]`, `ALL_MODELS` their concatenation; and `ENGINE = Engine()`, a process-wide singleton.

`class DatasetBundle`

A dataclass bundling one dataset's loaded resources: `spec`, `index` (`InvertedIndex`), `retrievers` (name → retriever, only those available), `refiner` (`QueryRefiner`).

`class Engine`

```
__init__(self)
```

Initializes an empty `_bundles` cache so each dataset's indexes load once and are reused.

`_bundle(self, dataset_key: str) -> _DatasetBundle`

Lazily loads/caches a dataset's resources. Resolves the spec, opens the `InvertedIndex` (raising `FileNotFoundError` with a build hint if absent), always wires up `tfidf` and `bm25`, adds a BERT `DenseRetriever` only if its `VectorStore` exists, and `Word2Vec` only if both its `VectorStore` and `Word2VecModel` artifacts exist. Builds a `QueryRefiner`, caches the bundle, returns it.

`available_models(self, dataset_key: str) -> list[str]`

Returns usable model ids: the single models present, plus the two hybrids when at least two base retrievers are available.

`refine(self, dataset_key: str, query: str, *, do_spell=True, do_expand=True, use_history=True, history: Optional[list[str]] = None) -> RefinementResult`

Delegates to the dataset's `QueryRefiner.refine`, passing the spell/expand/history flags plus prior history.

`suggest(self, dataset_key: str, query: str, history: Optional[list[str]] = None) -> list[str]`

Delegates to `QueryRefiner.suggest`, returning query suggestions.

`search(self, dataset_key: str, model: str, query: str, *, top_k: int = 10, bm25_k1: Optional[float] = None, bm25_b: Optional[float] = None, refine_opts: Optional[dict] = None, history: Optional[list[str]] = None, hybrid_opts: Optional[dict] = None) -> dict`

The main dispatcher. Loads the bundle, optionally refines the query (when `refine_opts["enabled"]`) to produce the effective raw query and tokens, builds a `QueryContext` via `build_context` (injecting BM25 params) and overrides its tokens with refined tokens if present. Dispatches to a single retriever, `_parallel`, or `_serial` (else `ValueError`), rennumbers ranks 1..n, and returns a response dict (dataset/model/query, effective query, params, refinement `to_dict` if any, serialized results).

`_default_pair(self, b: _DatasetBundle) -> list[str]`

Chooses a default two-model set for hybrids, preferring `bm25` + a dense model in priority order `["bm25", "bert", "word2vec", "tfidf"]`. Returns the first two available.

`_parallel(self, b, ctx, top_k, opts) -> list[SearchResult]`

Sets up/runs the parallel hybrid: picks names from `opts["models"]` or `_default_pair`, filters to available, raises `ValueError` if fewer than two remain, then calls `parallel_hybrid` with the chosen retrievers, `fusion` (default `weighted`), optional `weights`, and a candidate `pool` (default `max(100, top_k*5)`).

`_serial(self, b, ctx, top_k, opts) -> list[SearchResult]`

Sets up/runs the serial hybrid: derives stage-1/stage-2 from `_default_pair` (overridable via `opts`), validates both available (else `ValueError`), and calls `serial_hybrid` with `candidate_k` (default 100).

Section E — Evaluation Layer

`ir_core/eval/metrics.py`

Purpose: Implements the standard IR effectiveness metrics (requirement #8). Each function takes a ranked list of `doc_ids` (best first) plus the query's relevance judgments `qrel = {doc_id: relevance}`; binary metrics treat any `relevance > 0` as relevant, while `nDCG` uses the graded relevance value as the gain.

```
def precision_at_k(ranked: list[str], qrel: dict[str, int], k: int = 10) -> float
```

Precision@k: the fraction of the top-k ranked docs that are relevant. Counts hits with `qrel.get(d, 0) > 0` among the first k, returns `hits / k` (0.0 if `k <= 0`). The denominator is always k, so a short ranking is penalized against the full cutoff.

```
def recall(ranked: list[str], qrel: dict[str, int], k: int | None = None) -> float
```

Recall: the fraction of all relevant docs that appear. `total_rel` counts qrel entries > 0 (0.0 if none). The pool is the first k docs (or the whole ranking if k is None); returns `hits / total_rel`.

```
def average_precision(ranked: list[str], qrel: dict[str, int], k: int | None = None) -> float
```

Average Precision (AP) for one query — the mean of precision values at each rank where a relevant doc is found. Iterating with 1-based index `i`, each relevant hit adds the running precision `hits / i`; the sum is divided by `total_rel` (0.0 if none). Averaging AP across queries yields MAP.

```
def dcg(gains: list[float]) -> float
```

Discounted Cumulative Gain for an ordered gain list: `sum(g / log2(i + 1))` over 1-based positions. The discount means deeper-rank gains contribute less. Shared by `ndcg_at_k` for both actual and ideal rankings.

```
def ndcg_at_k(ranked: list[str], qrel: dict[str, int], k: int = 10) -> float
```

Normalized DCG at cutoff k with graded gains. `gains` is the graded relevance of the top-k docs (0 for unjudged); `ideal` is the top-k graded relevances sorted descending. Returns `dcg(gains) / dcg(ideal)`, or 0.0 when IDCG is 0 — a score in `[0, 1]`.

```
def evaluate_query(ranked: list[str], qrel: dict[str, int], p_at: int = 10, ndcg_at: int = 10, ap_depth: int | None = None, recall_depth: int | None = None) -> dict[str, float]
```

Scores a single query with all four metrics, returning a dict keyed "P@10", "Recall", "AP", "nDCG@10". The cutoffs control each metric (`ap_depth/recall_depth` None = full ranking). The dict keys are fixed strings regardless of the actual cutoff values.

```
def aggregate(per_query: list[dict[str, float]]) -> dict[str, float]
```

Averages each per-query metric across queries. The mean of "AP" is reported as "MAP"; `Recall`, `P@10`, `nDCG@10` are averaged under their own names; `num_queries` records the count. Returns all-zeros with `num_queries: 0` for empty input.

[ir_core/eval/evaluate.py](#)

Purpose: The evaluation runner (requirement #8) that executes a model over a dataset's judged queries, scores each ranking against the qrels via `metrics`, and aggregates to MAP/Recall/P@10/nDCG@10. It also supports a before/after comparison of query refinement.

```
def evaluate_model(dataset_key: str, model: str, *, num_queries: Optional[int] = 50,
eval_depth: int = 100, bm25_k1: Optional[float] = None, bm25_b: Optional[float] = None,
refine_opts: Optional[dict] = None, hybrid_opts: Optional[dict] = None, only_judged: bool =
True, progress=None) -> dict
```

Evaluates one model on one dataset. Resolves the spec, loads qrels/queries, optionally filters to only queries with qrels, caps to `num_queries`. For each query it calls `ENGINE.search(...)` with the given depth, BM25 params, refinement and hybrid options, extracts the ranked doc-ids, scores them with `metrics.evaluate_query`, and accumulates per-query results plus a rounded `details` record (with an optional `progress(i, n, qid)` callback). Returns a dict with the dataset/model, a `refinement_enabled` flag, BM25 `params`, the aggregated `metrics`, and `per_query` details.

```
def compare_refinement(dataset_key: str, model: str, *, num_queries: Optional[int] = 50,
eval_depth: int = 100, bm25_k1=None, bm25_b=None, hybrid_opts=None, progress=None) -> dict
```

Runs the model twice to quantify refinement's effect. `before` disables refinement; `after` enables it (`{"enabled": True, "spell": True, "expand": True, "history": False}`). Computes the per-metric `delta` (after – before, 4 dp) over MAP/Recall/P@10/nDCG@10 and returns `{dataset, model, before, after, delta}`.

Section F — Command-Line Scripts

`scripts/build_indexes.py`

Purpose: CLI that builds every representation/index (inverted, word2vec, bert) for a dataset (requirement #3 plus the index side of #2), streaming the preprocessed corpus so it works on full multi-GB files without loading them into RAM. Inserts the project root onto `sys.path` so `ir_core` resolves when run as a script.

```
def build(dataset_key: str, models: list[str], limit, w2v_params: Word2VecParams,
bert_batch: int, w2v_train_limit=None)
```

Builds the requested indexes. Resolves the spec, normalizes a falsy `limit` to `None` (ALL), defaults `w2v_train_limit` to `limit` (so Word2Vec can train on a cheap sample yet embed the full corpus). Builds the inverted index if requested; trains and saves Word2Vec then builds its embedding index; builds the BERT index with `bert_batch`. Each step is timed and logged.

```
def main()
```

Argparse entry point. Defines the flags, parses them, splits `--models`, constructs `Word2VecParams`, converts a `--w2v-train-limit` of 0 to `None`, and calls `build(...)`. Arguments: `--dataset` (required: `argsme | argsme_sample`); `--models` (default `"inverted,word2vec,bert"`); `--limit` (int, default 50000; 0 = ALL); `--w2v-train-limit` (int, default `None`); `--w2v-dim` (100), `--w2v-epochs` (3), `--w2v-min-count` (2); `--bert-batch` (256). `__main__` calls `main()`.

`scripts/evaluate.py`

Purpose: CLI front-end for single-model evaluation (requirement #8) that prints MAP/Recall/P@10/nDCG@10 as JSON, optionally comparing the basic pipeline against the refined one. Inserts the project root onto `sys.path`.

```
def main()
```

Parses arguments, defines a `prog(i, n, qid)` progress callback (every 10th query plus the last), and dispatches. With `--compare` it calls `compare_refinement(...)`; otherwise it builds `refine_opts` (on when `--refine`, else off), calls `evaluate_model(...)`, and trims the result to `dataset/model/refinement_enabled/metrics`. Prints `json.dumps(out, indent=2)`. Arguments: `--dataset` (required); `--model` (default `bm25`); `--num-queries` (50); `--eval-depth` (100); `--k1/--b` (BM25 overrides); `--compare` (flag); `--refine` (flag). `__main__` calls `main()`.

`scripts/run_full_eval.py`

Purpose: CLI that evaluates every available model on a dataset and emits a Markdown table plus a JSON dump to populate `docs/REPORT.md` (requirement #8). Inserts the project root onto `sys.path` and forces UTF-8 stdout (so the delta sign doesn't crash cp1252 consoles). Constants `MODEL_ORDER` and `METRICS` fix the column/row ordering.

```
def md_table(rows: list[dict], cols: list[str], head: str) -> str
```

Renders row dicts into a GitHub-flavored Markdown table. The header is `head + cols`, then a separator, then each row reads `r['name']` plus each column formatted as `{:.4f}` for floats (else stringified; missing → empty). Returns the joined table.

```
def main()
```

Parses arguments, filters `MODEL_ORDER` to models available for the dataset, and for each runs `evaluate_model(...)` with refinement disabled, collecting a metrics row and the raw metrics. Prints the basic-pipeline table. With `--refine-compare` it runs `compare_refinement` for whichever of `bm25/tfidf` are available, appending before/after/delta rows. With `--out` it writes `{per_model, table, compare_table}` as JSON (UTF-8). Arguments: `--dataset` (required); `--num-queries` (49); `--eval-depth` (100); `--refine-compare` (flag); `--out` (default None). `__main__` calls `main()`.

`scripts/smoke_test.py`

Purpose: End-to-end smoke test on the built-in 5-doc sample corpus (no `ir_datasets` needed) that builds every representation, exercises each model plus both hybrids and query refinement — a quick install check. Inserts the project root onto `sys.path`.

```
def main()
```

Builds the sample indexes (`InvertedIndex(SAMPLE).build(...)`, then `train_word2vec` with small params, saves it, builds the word2vec and BERT embedding indexes). Clears `ENGINE._bundles` to force a reload, then for the query `"school condoms for students"` prints the available models and, for each, runs `ENGINE.search(..., top_k=2, hybrid_opts={"fusion": "weighted"})` reporting the top hit. Finally calls `ENGINE.refine(SAMPLE.key, "shcool condom", history=["teen pregnancy"])`, prints corrections and expansions, and prints `"SMOKE TEST PASSED"`. `__main__` calls `main()`.

Section G — Service-Oriented Architecture (SOA)

SOA design summary. The backend is six independently deployable FastAPI processes. Five are domain services — Preprocessing (8001), Indexing (8002), Retrieval (8003), Ranking & Evaluation (8004), and Query Refinement (8005) — each built via `common.make_app(...)` (so each auto-exposes `GET /health`) and each delegating to the shared `ir_core` library. The API Gateway (8000) is the only public entry point: it holds no business logic but routes/forwards every request to the owning service using the stdlib `post_json/get_json` clients, wraps downstream failures as HTTP 502 via `_safe`, exposes a `/services` discovery roll-up, and composes services for `/search_refined` (calls Refinement then feeds the refined query into Retrieval). `run_all.py` launches all six as separate `uvicorn` subprocesses (services first, gateway last, staggered by 1s) and terminates them all on Ctrl-C/SIGINT.

services/common.py

Purpose: Shared helpers for the microservices: a FastAPI app factory (with a built-in health route) and a tiny stdlib `urllib`-based HTTP client used by the gateway to call downstream services over REST, keeping services loosely coupled with no extra dependency.

```
def make_app(title: str) -> FastAPI
```

Creates and returns a `FastAPI` instance titled `title` (version `"1.0.0"`) with a nested `GET /health` route returning `{"status": "ok", "service": title}`. Used by all five services and the gateway.

```
def post_json(url: str, payload: dict, timeout: float = 600.0) -> Any
```

Sends a JSON POST to `url`. JSON-encodes `payload` to UTF-8, builds a POST `Request` with `Content-Type: application/json`, opens it with the given timeout (default 600s for heavy retrieval/eval calls), and returns the decoded JSON response. The gateway's primary forwarding mechanism.

```
def get_json(url: str, timeout: float = 60.0) -> Any
```

Sends a GET to `url` (default 60s) and returns the parsed JSON. Used for health/discovery and read-only lookups (datasets, status, models).

services/gateway.py

Purpose: The API Gateway (port 8000) — the single public entry point implementing the Gateway/Facade pattern. It routes each call to the responsible downstream service over REST and composes multiple services when needed, so clients (including the UI) only talk to the gateway. Run with `uvicorn services.gateway:app --port 8000`. Globals: `app = make_app("api-gateway")`, `S = SERVICE_URLS`.

```
def _safe(fn, *a, **kw)
```

Invokes `fn(*a, **kw)` and, on any exception, re-raises it as `HTTPException(status_code=502, detail="downstream error: ...")` — a clean 502 instead of an uncaught 500.

```
GET /services(handler services_health())
```

Service discovery: iterates `SERVICE_URLS` (skipping `gateway`), calls `GET {url}/health` (3s timeout) on each, returning `{"url": ..., ...health}` for reachable services and `{"url": ..., "status": "down", "error": ...}` for unreachable ones, keyed by service name.

```
GET /datasets(handler datasets())
```

Forwards to the indexing service: `_safe(get_json, f"{S['indexing']}/datasets")`.

GET /status/{dataset_key} (handler status(dataset_key: str))

Forwards to `GET {indexing}/status/{dataset_key}`, returning its build-status payload.

GET /models/{dataset_key} (handler models(dataset_key: str))

Forwards to `GET {retrieval}/models/{dataset_key}`, returning the available retrieval models.

Request models

- **SearchRequest(BaseModel)** — `dataset: str; model: str = "bm25"; query: str; top_k: int = 10; bm25_k1: Optional[float] = None; bm25_b: Optional[float] = None; refine_opts: Optional[dict] = None; history: Optional[list[str]] = None; hybrid_opts: Optional[dict] = None.`
- **RefineRequest(BaseModel)** — `dataset: str; query: str; spell: bool = True; expand: bool = True; history_personalize: bool = True; history: Optional[list[str]] = None.`
- **EvalRequest(BaseModel)** — `dataset: str; model: str = "bm25"; num_queries: Optional[int] = 50; eval_depth: int = 100; bm25_k1: Optional[float] = None; bm25_b: Optional[float] = None; refine_opts: Optional[dict] = None; hybrid_opts: Optional[dict] = None.`

POST /search (handler search(req: SearchRequest))

Forwards `req.model_dump()` to `POST {retrieval}/search`; returns the ranked results.

POST /refine (handler refine(req: RefineRequest))

Forwards to `POST {refinement}/refine`; returns the refinement result.

POST /suggest (handler suggest(req: RefineRequest))

Forwards to `POST {refinement}/suggest`; returns query suggestions.

POST /evaluate (handler evaluate(req: EvalRequest))

Forwards to `POST {ranking_eval}/evaluate`; returns effectiveness metrics.

POST /compare (handler compare(req: EvalRequest))

Forwards to `POST {ranking_eval}/compare`; returns before/after-refinement metrics.

POST /search_refined (handler search_refined(req: SearchRequest))

The composition endpoint orchestrating two services. It first calls `POST {refinement}/refine` with a refine payload, then copies the original request, replaces its `query` with `refinement.get("refined_raw")` or `req.query`, and calls `POST {retrieval}/search`. Attaches the refinement result under `result["refinement"]` and returns the combined object.

[services/preprocessing_service.py](#)

Purpose: Preprocessing Service (port 8001). Single responsibility: turn raw text into normalized tokens using the *same* pipeline applied to the corpus (requirement #4). Global: `app = make_app("preprocessing-service")`.

class ProcessRequest(BaseModel)

Fields: `text: str; do_stemming: bool = True; do_lemmatize: bool = False; remove_stopwords: bool = True.`

POST /process (handler process_text(req: ProcessRequest))

Calls `ir_core.text.preprocessing.process(...)` with the request flags and returns `{"text", "tokens", "num_tokens"}`.

services/indexing_service.py

Purpose: Indexing Service (port 8002). Owns the inverted index and vector stores: reports build status, exposes term/vocabulary lookups, and can trigger a (capped) build. Full-corpus builds are normally done offline via `scripts/build_indexes.py`. Global: `app = make_app("indexing-service")`.

GET /datasets (handler datasets())

Returns `{"datasets": list_datasets()}`.

GET /status/{dataset_key} (handler status(dataset_key: str))

Resolves the dataset, constructs an `InvertedIndex(spec)`, and reports `inverted_index/bert/word2vec` existence (word2vec requires both its vector store and `Word2VecModel`). If the inverted index exists, adds `num_docs` and `avgdl` (rounded). Returns the info dict.

GET /vocab/{dataset_key} (handler vocab(dataset_key: str, prefix: str = "", limit: int = 20))

Loads the `InvertedIndex`; if not built, returns `{dataset, "terms": [], "error": "index not built"}`. Otherwise returns `{dataset, "terms": idx.vocab_sample(prefix, limit=limit)}`.

class BuildRequest(BaseModel)

Fields: `dataset_key: str; limit: Optional[int] = 5000`.

POST /build (handler build(req: BuildRequest))

Resolves the spec, calls `InvertedIndex(spec).build(limit=req.limit)`, returns `{"dataset", "built": True, **meta}`.

services/retrieval_service.py

Purpose: Retrieval Service (port 8003). Owns query matching & ranking (requirement #6): given a dataset, model and query it returns ranked documents, delegating to the shared `ENGINE`. BM25 `k1/b` are accepted per request, and hybrid options pass through. Global: `app = make_app("retrieval-service")`.

class SearchRequest(BaseModel)

Same shape as the gateway's `SearchRequest` (`dataset, model="bm25", query, top_k=10, bm25_k1, bm25_b, refine_opts, history, hybrid_opts`).

GET /models/{dataset_key} (handler models(dataset_key: str))

Returns `{"dataset": dataset_key, "models": ENGINE.available_models(dataset_key)}`.

POST /search (handler search(req: SearchRequest))

Delegates to `ENGINE.search(...)` with all request fields and returns the ranked list.

services/ranking_eval_service.py

Purpose: Ranking & Evaluation Service (port 8004). Owns offline effectiveness measurement (requirement #8): MAP, Recall, P@10, nDCG@10 over a dataset's qrels, plus a before/after-refinement comparison. Global: `app = make_app("ranking-eval-service")`.

class EvalRequest(BaseModel)

Fields: `dataset`; `model="bm25"`; `num_queries: Optional[int] = 50`; `eval_depth: int = 100`; `bm25_k1`; `bm25_b`; `refine_opts`; `hybrid_opts`.

POST /evaluate (handler evaluate(req: EvalRequest))

Calls `evaluate_model(...)` with the request fields and returns the metric dict.

POST /compare (handler compare(req: EvalRequest))

Calls `compare_refinement(...)` to evaluate the model with refinement OFF then ON (note: `refine_opts` is not forwarded here) and returns the comparison.

services/query_refinement_service.py

Purpose: Query Refinement Service (port 8005). Owns requirement #5: spelling correction, WordNet synonym expansion, query suggestion, and history-based personalization, delegating to `ENGINE`. Global: `app = make_app("query-refinement-service")`.

class RefineRequest(BaseModel)

Fields: `dataset`; `query`; `spell=True`; `expand=True`; `history_personalize=True`; `history: Optional[list[str]] = None`.

class SuggestRequest(BaseModel)

Fields: `dataset`; `query`; `history: Optional[list[str]] = None`.

POST /refine (handler refine(req: RefineRequest))

Calls `ENGINE.refine(...)` and returns `r.to_dict()` (including `refined_raw`).

POST /suggest (handler suggest(req: SuggestRequest))

Returns `{"query": req.query, "suggestions": ENGINE.suggest(...)}`.

services/run_all.py

Purpose: Dev launcher that starts every microservice plus the gateway, each as its own uvicorn subprocess, so the whole SOA backend comes up with `python -m services.run_all` (Ctrl-C to stop). `SERVICES` is an ordered list of `(target, port)` tuples (services on 8001-8005, gateway last on 8000).


```
def main()
```

Iterates `SERVICES`, launching each via `subprocess.Popen([sys.executable, "-m", "uvicorn", target, "--host", "127.0.0.1", "--port", str(port), "--log-level", "warning"])`, collecting handles, printing each URL, sleeping 1.0s between launches (staggered startup). Prints the gateway URL, defines a nested `shutdown(*_)` that terminates all children and exits, registers it on `SIGINT`, and `wait()`s on each process (with a `KeyboardInterrupt` fallback).

```
nested shutdown(*_)
```

Cleanup closure: prints a message, calls `p.terminate()` on every launched process, and exits with status 0.

Section H — Root Utility Scripts

preprocessing.py

Purpose: Standalone NLTK-based text-preprocessing module (the original/notebook-derived pipeline used to generate the corpus JSON). On import it ensures required NLTK resources are present and prepares shared singletons (`_STEMMER`, `_LEMMATIZER`, `_STOPWORDS_EN`).

```
def _get_wordnet_pos(treebank_tag)
```

Maps a Penn-Treebank POS tag to the WordNet POS constant for lemmatization (empty tag → noun; `J`→adj, `N`→noun, `V`→verb, `R`→adv; default noun).

```
def preprocess_text(text, language='english', lowercase=True, remove_punctuation=True,
remove_stopwords=True, do_stemming=False, do_lemmatize=False, do_pos_tag=False,
do_spell_check=False, use_spacy=False)
```

The flexible preprocessing pipeline. Returns `[]` for empty input, then: optional lowercasing; optional punctuation removal; `word_tokenize`; keep only alphabetic tokens; optional stopwords removal; POS tagging (if `do_pos_tag/do_lemmatize`); optional lemmatization; optional Porter stemming (after lemmatization); optional spell-check via `pyspellchecker` (skipped if missing). Returns `{ 'tokens', 'pos_tags' }` when `do_pos_tag`, else the token list. (`use_spacy` is accepted but unused.)

```
def clean_and_preprocess(raw_text)
```

Backward-compatible wrapper calling `preprocess_text` with lowercase + punctuation removal + stopwords removal + stemming, returning the token list. `__main__` demonstrates both functions.

process_docs.py

Purpose: Script that iterates an `ir_datasets` dataset's documents, preprocesses each with `preprocessing.preprocess_text`, and writes `{"doc_id", "raw_text", "tokens"}` records to a JSON file. Imports `ir_win_fix` first.

```
def _doc_text(doc)
```

Returns the best available text for an `ir_datasets` document: prefers `doc.default_text()`, else the first present of `text/body/title`, else `''`.

```
def process_dataset_to_json(dataset_name, out_path, limit=1000, keep_raw=True,
**preprocess_kwargs)
```

Loads `dataset_name`, iterates `docs_iter()`, and for each doc computes `raw_text` and `tokens`, appending a record (plus `raw_text` when `keep_raw`). Prints progress every 1000 docs, stops at `limit` (falsy = all), writes UTF-8 JSON (`indent=2`), and returns `out_path`.

```
def _parse_args()
```

Builds/returns parsed CLI args: `--dataset` (default the Args.me id), `--out` (None), `--limit` (1000; 0 = all), `--no-raw` (flag), `--no-stemming` (flag). `__main__` derives a default `out_path` from a dataset slug when `--out` is omitted and calls `process_dataset_to_json`.

check_ready.py

Purpose: Small smoke-test script verifying the Args.me dataset is downloaded and usable. Imports `ir_win_fix` before `ir_datasets`. A top-level script (no functions): loads the Args.me dataset, prints `docs_count()`, pulls one query, and prints its id and text (via `getattr` fallbacks); any exception is caught and printed.

download_dataset.py

Purpose: Script that pre-fetches/materializes an `ir_datasets` dataset (documents, queries, qrels) so later runs hit a warm cache.

```
def download_dataset(dataset_name)
```

Loads `dataset_name`, then forces materialization of each component: prints `docs_count()`, counts queries and qrels via `sum(1 for _ in ...)`, printing progress/counts. Any exception is caught and reported. The module calls `download_dataset("argsme/1.0/touche-2020-task-1/uncorrected")` at run time.

ir_win_fix.py

Purpose: A Windows compatibility shim for an `ir_datasets` download bug. `ir_datasets` creates its download target with `tempfile.NamedTemporaryFile(delete=False)` but never closes the handle; later `os.replace('<tmp>.tmp', '<tmp>')` fails on Windows (you cannot replace a file with an open handle → `WinError 5/WinError 32`). Must be imported *before* loading any dataset; importing it applies the patch.

```
class _TempfileShim
```

A proxy standing in for the real `tempfile` module inside `ir_datasets.util.download`, ensuring named temp files release their OS handle immediately so Windows can overwrite them.

```
NamedTemporaryFile(self, *args, **kwargs)
```

Calls the real `NamedTemporaryFile`, immediately `close()`s the result (releasing the Windows handle; the file persists because callers pass `delete=False`), and returns the closed file object so the subsequent `os.replace()` succeeds.

```
__getattr__(self, name)
```

Delegates every other attribute access to the real `tempfile` module, so the shim behaves identically except for `NamedTemporaryFile`.

Module-level patch: guarded by an idempotency check, it sets `_download.tempfile = _TempfileShim()`, monkey-patching only the `download` module's view of `tempfile` (the global `tempfile` is untouched).

Section I — User Interface

ui/app.py

Purpose: A single-page Streamlit application that is the user-facing search interface (requirement #9). It lets the user pick a dataset and representation model, tune BM25 parameters and hybrid options live, toggle query refinement, and view ranked results as cards with score bars, query-term highlighting, per-model component scores, and inline full-document expanders. It also provides an Evaluation tab (MAP/Recall/P@10/nDCG@10 plus before/after-refinement comparison) and a Help tab, and can run against the engine in-process or against the REST gateway via a sidebar toggle.

Module-level configuration

- **sys.path insertion:** prepends the project root so `ir_core/services` import when Streamlit runs the file directly.
- **st.set_page_config(...):** title "IR System", icon , layout="wide", called once at import.
- **MODEL_LABELS (dict):** internal model keys → human-readable selectbox labels. Used as the model picker's `format_func`, in the header caption, spinners, and Help tab.
- **MODEL_HELP (dict):** model keys → one-line descriptions, shown under the model selectbox and in Help.
- **EXAMPLES (dict):** curated clickable example queries per dataset key. Rendered as buttons in Search and listed in Help.
- **CSS (str):** an inline `<style>` block styling the result cards (`.result-card`, `.rank-badge`, `.docid`, `.score-val`, `.score-bar`, `.comp-badge`, `.snippet`, `mark`, `.pill`), injected once.
- **Session defaults:** seeds `st.session_state` keys via `setdefault` (`history=[]`, `query_box=""`, `do_search=False`, `last_resp=None`, `last_latency=0.0`).

class Backend (abstract interface)

Defines the backend contract as stub methods: `datasets()`, `models(ds)`, `status(ds)`, `search(**kw)`, `refine(**kw)`, `evaluate(**kw)`, `compare(**kw)`. Subclassed by the two backends.

class InProcessBackend(Backend)

Calls `ir_core` directly, so it works with no servers running. - `__init__`: imports `ENGINE` (as `self.E`) and `evaluate_model/compare_refinement`. - `datasets`: returns `list_datasets()`. - `models(ds)`: returns `self.E.available_models(ds)`. - `status(ds)`: builds a status dict by constructing `InvertedIndex` and `VectorStore` and calling `.exists()`; adds `num_docs` if the index exists. - `search(**kw)`: pops `dataset/model/query` and forwards the rest to `self.E.search(...)`. - `refine(dataset, query, history=None, **flags)`: calls `self.E.refine(...)` mapping `spell/expand/history_personalize` to `do_spell/do_expand/use_history`; returns `r.to_dict()`. - `evaluate/compare`: pop `dataset/model` and delegate to the eval helpers.

class GatewayBackend(Backend)

Talks to the REST microservices (full SOA path). - `__init__(base)`: stores `base.rstrip("/")` and imports `get_json/post_json`. - `datasets/models/status`: GET the corresponding gateway endpoints. - `search/refine/evaluate/compare`: POST the kwargs to the corresponding endpoint.

`@st.cache_resource def get_backend(mode: str, gateway_url: str) -> Backend`

Returns a `GatewayBackend(gateway_url)` when `mode == "Gateway (SOA)"`, else `InProcessBackend()`. Cached so the backend (and its engine/clients) is built once and reused across reruns, keyed on `(mode, gateway_url)`.

```
def highlight(text: str, terms: list[str], limit: int | None = 600) -> str
```

HTML-escapes `text` and wraps matched query/expansion terms in `<mark>`. Returns `"<em>(no stored text)</em>"` if empty. If `limit` is not `None` and the text exceeds it, truncates to a preview snippet; **passing `limit=None` renders the complete document** (used by the full-document expander). Converts newlines to `<br>` and highlights unique terms of length ≥ 2 , longest-first, with a case-insensitive word-boundary regex.

```
def query_terms(query: str, refinement: dict | None) -> list[str]
```

Extracts alphabetic tokens from `query`; if a `refinement` dict is given, also appends the words of each `expansions` entry and the values of `corrections`. The result is fed into `highlight(...)`.

```
def trigger_search(q: str | None = None)
```

If `q` is provided, sets `st.session_state.query_box = q` (used by example/suggestion buttons); then sets `st.session_state.do_search = True` to request a search on the next rerun.

```
def refine_opts() -> dict
```

Returns `{"enabled": False}` when the refinement toggle is off; otherwise `{"enabled": True, "spell", "expand", "history"}` reflecting the sidebar checkboxes. Passed as `refine_opts=` to search/evaluate.

Page structure / tabs

Sidebar (top-to-bottom): title/caption; **Backend** radio (In-process / Gateway (SOA)) with a Gateway URL input when SOA is selected; a reachability check (`st.error + st.stop()` on failure); **Dataset** picker; an index-status caption (`■ N docs | idx ■/■ · bert ■/■ · w2v ■/■`); model-availability check (warns with the build command and stops if none); **Representation model** picker with help caption; **BM25 parameters** expander (`k1`, `b` sliders) for bm25/hybrids; **hybrid configuration** (parallel: models-to-fuse multiselect + fusion method; serial: stage-1/stage-2 pickers + candidate-pool slider); **Query refinement** toggle revealing spelling/synonym/history checkboxes; **top-k** slider.

Header: `■ Information Retrieval System` plus a caption naming the active dataset and model and whether refinement is on. Then three tabs: Search, Evaluation, Help.

Search tab: a query input + Search button; example-query buttons; live suggestions (when refinement is on); search execution (appends to history, times the call, runs `backend.search(...)`, stores `last_resp/last_latency`); results rendering — an optional refinement panel (corrections/expansions/effective query), a summary line, and per-result cards (rank badge, monospace doc-id, right-floated score, score bar, highlighted 600-char snippet, per-model component badges), each followed by a `■ Read full document` expander rendering the entire original document inline via `highlight(..., limit=None)`; and a search-history expander with a clear button.

Evaluation tab (requirement #8): controls for judged-query count and retrieval depth; an **Evaluate model** button (metric cards + a bar chart) and a **Compare before/after refinement** button (before→after metric cards with deltas + a dataframe and an explanatory caption).

Help tab: describes what Args.me retrieves and lists its example queries, lists every model with its description, and a Tips section.

End of code reference.